1. Importing libraries

In [179…
```
## Loading libraries
import os
import warnings
warnings.filterwarnings('ignore')
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import random
import ast
import lightgbm as lgb
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.metrics import roc_auc_score, log_loss



# ── Topic parser (defined here so EDA and feature engineering both use it) ──
def parse_topics(topic_str):
    """Parse topics_list (list of lists string) into a flat list of topic strings."
    if pd.isna(topic_str):
        return []
    try:
        parsed = ast.literal_eval(str(topic_str))
        flat = []
        for item in parsed:
            if isinstance(item, list):
                flat.extend([t.strip() for t in item])
            else:
                flat.append(str(item).strip())
        return flat
    except Exception:
        return [str(topic_str).strip()]
```

Setting seed

In [56]:
```
RANDOM_STATE = 42
random.seed(RANDOM_STATE)           # Python built-in random
np.random.seed(RANDOM_STATE)        # NumPy (used by sklearn internally)
os.environ['PYTHONHASHSEED'] = str(RANDOM_STATE)  # Python hash seed
print(f'✅ Imports complete | RANDOM_STATE={RANDOM_STATE} | Seeds set for: random,
```

```
✅ Imports complete | RANDOM_STATE=42 | Seeds set for: random, numpy, os.environ
```

In [58]:
```
Targets = ['adopted_within_07_days', 'adopted_within_90_days', 'adopted_within_120_
Target_labels = ['TX_07', 'TX_90', 'TX_120']  # for submission columns
ID_COL = 'ID'
n_splits = 5
```

2. Loading the data

In [60]:
```
train = pd.read_csv("Train.csv")
test = pd.read_csv("Test.csv")
prior = pd.read_csv("Prior.csv")
sample = pd.read_csv("SampleSubmission.csv")
```

In [99]:
```
def clean_trainer(val):
    """Extract first trainer from list-formatted string like ['TRA_abc'] or ['TRA_a
    if pd.isna(val):
        return val
    try:
        parsed = ast.literal_eval(str(val))
        if isinstance(parsed, list) and len(parsed) > 0:
            return parsed[0]  # take first trainer
    except:
        pass
    return str(val)

train['trainer'] = train['trainer'].apply(clean_trainer)
test['trainer']  = test['trainer'].apply(clean_trainer)
prior['trainer'] = prior['trainer'].apply(clean_trainer)

print("Trainer column after cleaning:")
print(train['trainer'].value_counts().head(10))
```

```
Trainer column after cleaning:
trainer
TRA_gertumxc    3049
TRA_suiifsur    2190
TRA_hyodnntj    2168
TRA_szrwyfzz    2028
TRA_rkvyofbh    1877
TRA_ubcgvofe    1456
TRA_kkzpfdtu     734
TRA_twwcfcum      24
TRA_dttdgplk      10
Name: count, dtype: int64
```

In [101…
```
print("Train shape:", train.shape)
print("Test shape:", test.shape)
print("Prior shape:", prior.shape)
```

```
Train shape: (13536, 39)
Test shape: (5621, 36)
Prior shape: (44882, 17)
```

In [103…
```
# How many rows vs unique farmers?
print("Prior unique Farmer IDs:", prior['ID'].nunique())
print("Train unique Farmer IDs:", train['ID'].nunique())

# How many prior farmers overlap with train?
overlap = prior['ID'].isin(train['ID']).sum()
print("Prior farmers that appear in Train:", overlap)
```

```
# Sample rows
print("\n=== PRIOR SAMPLE ===")
print(prior.head(5).to_string())

# What does topic list look like exactly?
print("\n=== TOPIC LIST SAMPLE (train) ===")
print(train['topics_list'].head(5).to_string())
```

```
Prior unique Farmer IDs: 44882
Train unique Farmer IDs: 13536
Prior farmers that appear in Train: 0

=== PRIOR SAMPLE ===
          ID  farmer_name training_day  gender registration       age   group_name
belong_to_cooperative      county    subcounty          ward  adopted_within_07_days
adopted_within_90_days  adopted_within_120_days  has_topic_trained_on       trainer
topics_list
0  ID_70GP6F  FAR_leopgvh   2024-01-03  Female       Manual  Above 35  GRP_yvpakgc
0  CNT_lpotuu  SUB_lpotuuf  WRD_lpotuufh                      0
0                        0                     0  TRA_szrwyfzz
['Ndume App', 'Poultry Feeding']
1  ID_IWQOWJ  FAR_vdcjfxm   2024-01-03  Female         Ussd  Above 35  GRP_yvpakgc
0  CNT_lpotuu  SUB_lpotuuf  WRD_lpotuufh                      0
0                        0                     0  TRA_szrwyfzz
['Ndume App', 'Poultry Feeding']
2  ID_Z3ES85  FAR_hfkybdg   2024-01-03  Female       Manual  Above 35  GRP_zmblxsw
0  CNT_fhdsoy  SUB_mdyljqn  WRD_atkhhvon                      0
0                        0                     1  TRA_rkvyofbh  ['Asili Fertilizer
(Organic)', 'Biosecurity In Poultry Farming', 'Feeding A Lactating Cow', 'How To Fee
d Kienyeji Chicken From 1 Day Old To Maturity', 'How To Feed Layers From 1 Day Old T
o Maturity', 'How To Rear Healthy Chicken With Biodeal.', 'Importance Of Choosing Th
e Right Seed Variety', 'Importance Of Vaccinations And Record', 'Pest And Disease Ma
nagement In Maize And Beans', 'Poultry And Dairy Feeding With Tyari Feeds', 'Poultry
Housing', 'Poultry Products', 'Record Keeping In Dairy', 'Unga Dairy Feeds', 'Hygien
e And Health Products']
3  ID_JNZM6R  FAR_hfkybdg   2024-01-03  Female       Manual  Above 35  GRP_psdrfni
0  CNT_fhdsoy  SUB_mdyljqn  WRD_atkhhvon                      0
0                        0                     1  TRA_rkvyofbh
['Poultry Products']
4  ID_BNJ1GU  FAR_hfkybdg   2024-01-03  Female       Manual  Above 35  GRP_psdrfni
1  CNT_fhdsoy  SUB_mdyljqn  WRD_atkhhvon                      0
0                        0                     1  TRA_rkvyofbh
['Record Keeping In Dairy']

=== TOPIC LIST SAMPLE (train) ===
0                    [['Ndume App', 'Poultry Feeding']]
1           [['Poultry Housing'], ['Poultry Housing']]
2    [['Asili Fertilizer (Organic)', 'Biosecurity I...
3    [['Poultry Products'], ['Record Keeping In Dai...
4                    [['Ndume App', 'Poultry Feeding']]
```

In [105…

```
print("=== TARGET BALANCE ===")
summary = []
for col in Targets:
    total    = len(train[col])
    adopted  = train[col].sum()
```

```
    not_adopted = total - adopted
    rate     = adopted / total
    ratio    = not_adopted / adopted  # how many 0s per 1
    summary.append({
        'Target':       col,
        'Total':        total,
        'Adopted (1)':  adopted,
        'Not Adopted (0)': not_adopted,
        'Adoption Rate': f'{rate*100:.2f}%',
        'Imbalance Ratio': f'{ratio:.0f}:1'
    })

display(pd.DataFrame(summary).set_index('Target'))
```

```
=== TARGET BALANCE ===
```

Target	Total	Adopted (1)	Not Adopted (0)	Adoption Rate	Imbalance Ratio
adopted_within_07_days	13536	153	13383	1.13%	87:1
adopted_within_90_days	13536	214	13322	1.58%	62:1
adopted_within_120_days	13536	302	13234	2.23%	44:1

In [107…

```
#Check missing values
print('=== MISSING VALUES IN TRAIN ===')
missing = train.isnull().sum()
print(missing[missing > 0] if missing.any() else 'No missing values')

print('\n=== MISSING VALUES IN TEST ===')
missing_test = test.isnull().sum()
print(missing_test[missing_test > 0] if missing_test.any() else 'No missing values'
```

```
=== MISSING VALUES IN TRAIN ===
No missing values

=== MISSING VALUES IN TEST ===
No missing values
```

In [109…

```
# Cardinality of categorical columns
cat_cols = ['gender', 'age', 'registration', 'group_name',
            'county', 'subcounty', 'ward', 'trainer']
print('=== CARDINALITY ===')
for col in cat_cols:
    if col in train.columns:
        print(f'{col:25s}: {train[col].nunique():5d} unique values')
```

```
=== CARDINALITY ===
gender                    :     2 unique values
age                       :     2 unique values
registration              :     2 unique values
group_name                :   864 unique values
county                    :     8 unique values
subcounty                 :    26 unique values
ward                      :    66 unique values
trainer                   :     9 unique values
```

In [111…

```python
def parse_topics(topic_str):
    """Parse topics_list string into a flat list of topic strings."""
    if pd.isna(topic_str):
        return []
    try:
        parsed = ast.literal_eval(str(topic_str))
        # Handle list of lists: [['topic1', 'topic2'], ['topic3']]
        if isinstance(parsed, list):
            flat = []
            for item in parsed:
                if isinstance(item, list):
                    flat.extend([t.strip() for t in item])
                else:
                    flat.append(str(item).strip())
            return flat
    except Exception:
        pass
    # Fallback: treat as plain string
    return [str(topic_str).strip()]


def engineer_topic_features(df, topic_col='topics_list'):
    """Extract numeric features from the topics_list column."""
    topics_parsed = df[topic_col].apply(parse_topics)

    df = df.copy()
    df['topic_total_count']  = topics_parsed.apply(len)
    df['topic_unique_count'] = topics_parsed.apply(lambda x: len(set(x)))
    df['topic_sessions']     = df[topic_col].apply(
        lambda x: len(ast.literal_eval(str(x))) if pd.notna(x) else 0
    )
    df['topic_repeat_rate']  = df['topic_total_count'] - df['topic_unique_count']

    # Get all topics across train+test for one-hot encoding
    return df, topics_parsed


def get_top_topics(train_parsed, top_n=20):
    """Find the most common topics in the training set."""
    from collections import Counter
    all_topics = [t for topics in train_parsed for t in topics]
    counter    = Counter(all_topics)
    top_topics = [t for t, _ in counter.most_common(top_n)]
    print(f'Top {top_n} topics:')
    for t, c in counter.most_common(top_n):
        print(f'  {c:5d}x  {t}')
```

```
    return top_topics


def add_topic_dummies(df, topics_parsed, top_topics):
    """Add binary columns for each top topic."""
    df = df.copy()
    for topic in top_topics:
        col_name = 'topic_' + topic.lower().replace(' ', '_').replace('(', '').repl
        df[col_name] = topics_parsed.apply(lambda x: int(topic in x))
    return df


# Apply to train
train, train_topics_parsed = engineer_topic_features(train)
top_topics = get_top_topics(train_topics_parsed, top_n=20)
train = add_topic_dummies(train, train_topics_parsed, top_topics)

# Apply to test
test, test_topics_parsed = engineer_topic_features(test)
test = add_topic_dummies(test, test_topics_parsed, top_topics)

print(f'\nTrain shape after topic features: {train.shape}')
```

```
Top 20 topics:
   1657x  The Benefits Of Ndume App
   1294x  Herd Health Management.
   1247x  Biosecurity In Poultry Farming
   1246x  Feeding A Lactating Cow
   1193x  Calf Feeding
   1153x  Poultry Management Practises.
   1091x  Poultry Health Management.
    966x  Dairy Feed Management
    934x  Poultry Diseases And Biosecurity
    930x  Record Keeping In Poultry
    894x  Dairy Health Management
    855x  Milking Hygiene
    855x  Poultry Health Management
    854x  Poultry Management Practices
    850x  How To Succeed In Breeding
    823x  Diseases In Dairy Farming
    823x  Poultry Management Practises..
    821x  Poultry Housing
    806x  Dairy Housing
    788x  Record Keeping In Dairy

Train shape after topic features: (13536, 39)
```

Adoption rates (%) per category

In [114…

```
for col in ['gender', 'age', 'registration', 'belong_to_cooperative']:
    print(f"\n=== {col} ===")
    display(train.groupby(col)[TARGETS].mean().round(3) * 100)
```

```
=== gender ===
```

gender	adopted_within_07_days	adopted_within_90_days	adopted_within_120_days
Female	1.1	1.5	2.2
Male	1.3	1.8	2.4

```
=== age ===
```

age	adopted_within_07_days	adopted_within_90_days	adopted_within_120_days
Above 35	1.1	1.5	2.2
Below 35	1.4	2.0	2.5

```
=== registration ===
```

registration	adopted_within_07_days	adopted_within_90_days	adopted_within_120_days
Manual	0.7	0.8	1.0
Ussd	1.4	2.1	3.0

```
=== belong_to_cooperative ===
```

belong_to_cooperative	adopted_within_07_days	adopted_within_90_days	adopted_within_120_
0	0.9	1.4	
1	2.4	2.6	

Gender: Males adopt slightly more than females across all time windows (1.3% vs 1.1% at 7 days, 1.8% vs 1.5% at 90 days). The difference is small though

Age: Younger farmers (Below 35) adopt at a consistently higher rate than older ones (1.4% vs 1.1% at 7 days, 2.5% vs 2.2% at 120 days). Moderate signal.

Registration method: USSD-registered farmers adopt at almost double the rate of manually registered farmers across all windows (1.4% vs 0.7% at 7 days, 3.0% vs 1.0% at 120 days). This is a strong predictive feature.

Cooperative membership: This is the strongest signal of all. Farmers who belong to a cooperative adopt at nearly 3x the rate of non-members at 7 days (2.4% vs 0.9%), and the gap widens over time (3.6% vs 2.0% at 120 days). Cooperative members likely have peer support, shared resources, and social accountability that encourages them to try new practices. This should be one of your most important features.

In [117…

```
print(train['trainer'].value_counts().head(20))
print("\nRows where trainer looks like a list:")
print(train[train['trainer'].str.startswith('[')]['trainer'].value_counts().head(10
print("\nTotal multi-trainer rows:", train['trainer'].str.startswith('[').sum())
```

```
trainer
TRA_gertumxc    3049
TRA_suiifsur    2190
TRA_hyodnntj    2168
TRA_szrwyfzz    2028
TRA_rkvyofbh    1877
TRA_ubcgvofe    1456
TRA_kkzpfdtu     734
TRA_twwcfcum      24
TRA_dttdgplk      10
Name: count, dtype: int64

Rows where trainer looks like a list:
Series([], Name: count, dtype: int64)

Total multi-trainer rows: 0
```

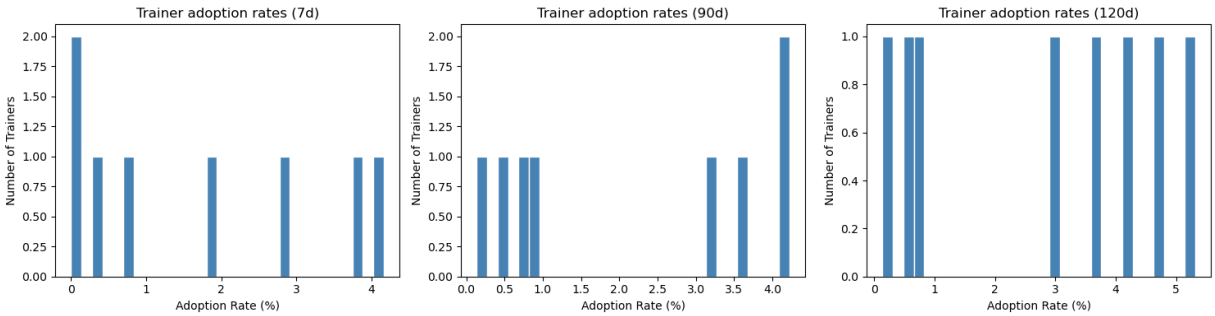
In [121…

```python
# ── 3.5 Trainer Effectiveness ──────────────────────────────
# Some trainers may be significantly better at driving adoption
trainer_stats = train.groupby('trainer').agg(
    farmer_count=('ID', 'count'),
    rate_07=('adopted_within_07_days', 'mean'),
    rate_90=('adopted_within_90_days', 'mean'),
    rate_120=('adopted_within_120_days', 'mean')
).reset_index()

# Only look at trainers with enough farmers to be meaningful
trainer_stats = trainer_stats[trainer_stats['farmer_count'] >= 20].copy()
trainer_stats[['rate_07','rate_90','rate_120']] *= 100

print(f'Trainers with 20+ farmers: {len(trainer_stats)}')
print('\nTop 10 trainers by 7-day adoption rate:')
display(trainer_stats.sort_values('rate_07', ascending=False).head(10)
        .round(3).reset_index(drop=True))

print('\nAdoption rate distribution across trainers:')
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for i, (col, label) in enumerate(zip(['rate_07','rate_90','rate_120'], ['7d','90d',
    axes[i].hist(trainer_stats[col], bins=30, color='steelblue', edgecolor='white')
    axes[i].set_title(f'Trainer adoption rates ({label})')
    axes[i].set_xlabel('Adoption Rate (%)')
    axes[i].set_ylabel('Number of Trainers')
plt.tight_layout()
plt.show()
```

```
Trainers with 20+ farmers: 8

Top 10 trainers by 7-day adoption rate:
```

	trainer	farmer_count	rate_07	rate_90	rate_120
0	TRA_twwcfcum	24	4.167	4.167	4.167
1	TRA_kkzpfdtu	734	3.815	4.223	5.313
2	TRA_rkvyofbh	1877	2.877	3.197	3.676
3	TRA_szrwyfzz	2028	1.923	3.600	4.635
4	TRA_hyodnntj	2168	0.830	0.876	2.906
5	TRA_gertumxc	3049	0.361	0.689	0.787
6	TRA_suiifsur	2190	0.091	0.137	0.137
7	TRA_ubcgvofe	1456	0.000	0.412	0.618

```
Adoption rate distribution across trainers:
```

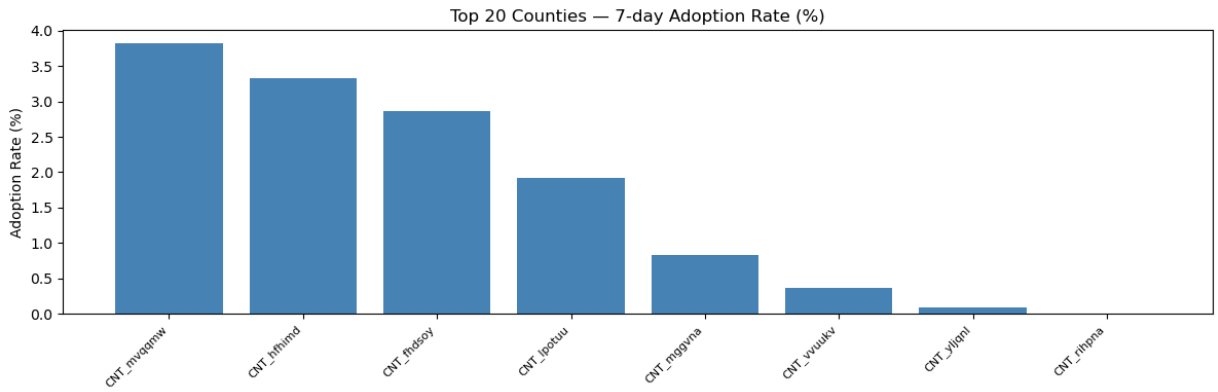


In [123…

```python
#  3.6 Geographic Adoption Patterns
county_stats = train.groupby('county').agg(
    farmer_count=('ID', 'count'),
    rate_07=('adopted_within_07_days', 'mean'),
    rate_90=('adopted_within_90_days', 'mean'),
    rate_120=('adopted_within_120_days', 'mean')
).reset_index()

county_stats = county_stats[county_stats['farmer_count'] >= 20].copy()
county_stats[['rate_07','rate_90','rate_120']] *= 100

print(f'Counties with 20+ farmers: {len(county_stats)}')
print('\nTop 10 counties by 7-day adoption rate:')
display(county_stats.sort_values('rate_07', ascending=False).head(10)
        .round(3).reset_index(drop=True))

# Visualise spread
fig, ax = plt.subplots(figsize=(12, 4))
county_sorted = county_stats.sort_values('rate_07', ascending=False).head(20)
ax.bar(range(len(county_sorted)), county_sorted['rate_07'], color='steelblue')
ax.set_xticks(range(len(county_sorted)))
ax.set_xticklabels(county_sorted['county'], rotation=45, ha='right', fontsize=8)
ax.set_title('Top 20 Counties: 7-day Adoption Rate (%)')
ax.set_ylabel('Adoption Rate (%)')
plt.tight_layout()
plt.show()
```

```
Counties with 20+ farmers: 8

Top 10 counties by 7-day adoption rate:
```

	county	farmer_count	rate_07	rate_90	rate_120
0	CNT_mvqqmw	734	3.815	4.223	5.313
1	CNT_hfhimd	30	3.333	3.333	3.333
2	CNT_fhdsoy	1887	2.862	3.180	3.657
3	CNT_lpotuu	2028	1.923	3.600	4.635
4	CNT_mggvna	2168	0.830	0.876	2.906
5	CNT_vvuukv	3049	0.361	0.689	0.787
6	CNT_yljqnl	2190	0.091	0.137	0.137
7	CNT_rihpna	1450	0.000	0.414	0.621



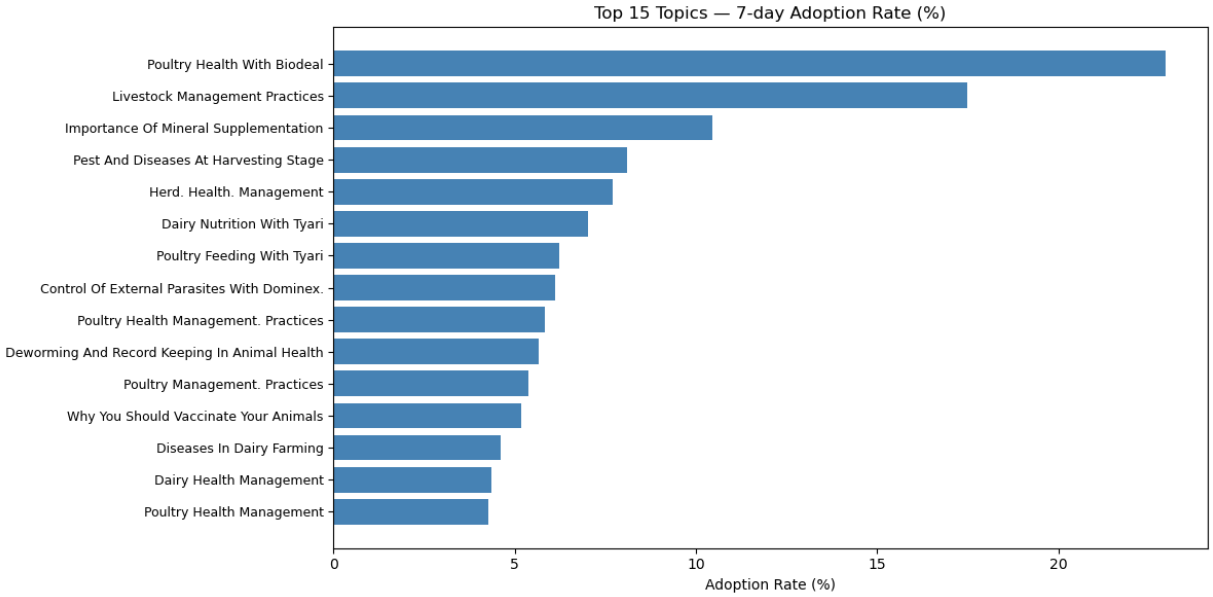
In [127…

```python
# ── 3.7 Topic Adoption Rates ──────────────────────────────
# Parse topics now (before full feature engineering) just for EDA
import ast
from collections import Counter

def parse_topics_flat(topic_str):
    if pd.isna(topic_str): return []
    try:
        parsed = ast.literal_eval(str(topic_str))
        flat = []
        for item in parsed:
            if isinstance(item, list): flat.extend([t.strip() for t in item])
            else: flat.append(str(item).strip())
        return list(set(flat))  # deduplicate within farmer
    except: return []

train_topics = train['topics_list'].apply(parse_topics_flat)

# Build topic x adoption dataframe
all_topics = list(set(t for topics in train_topics for t in topics))
topic_records = []
for topic in all_topics:
    mask  = train_topics.apply(lambda x: topic in x)
```

```
    count = mask.sum()
    if count < 20: continue  # skip rare topics
    topic_records.append({
        'topic':    topic,
        'count':    count,
        'rate_07':  train.loc[mask, 'adopted_within_07_days'].mean() * 100,
        'rate_90':  train.loc[mask, 'adopted_within_90_days'].mean() * 100,
        'rate_120': train.loc[mask, 'adopted_within_120_days'].mean() * 100,
    })

topic_df = pd.DataFrame(topic_records).sort_values('rate_07', ascending=False)
print('Top 15 topics by 7-day adoption rate (min 20 farmers):')
display(topic_df.head(15).round(3).reset_index(drop=True))

# Plot
fig, ax = plt.subplots(figsize=(12, 6))
top15 = topic_df.head(15)
ax.barh(range(len(top15)), top15['rate_07'], color='steelblue')
ax.set_yticks(range(len(top15)))
ax.set_yticklabels(top15['topic'], fontsize=9)
ax.invert_yaxis()
ax.set_title('Top 15 Topics — 7-day Adoption Rate (%)')
ax.set_xlabel('Adoption Rate (%)')
plt.tight_layout()
plt.show()
```

```
Top 15 topics by 7-day adoption rate (min 20 farmers):
```

	topic	count	rate_07	rate_90	rate_120
0	Poultry Health With Biodeal	74	22.973	24.324	24.324
1	Livestock Management Practices	40	17.500	17.500	17.500
2	Importance Of Mineral Supplementation	239	10.460	10.879	10.879
3	Pest And Diseases At Harvesting Stage	37	8.108	8.108	8.108
4	Herd. Health. Management	481	7.692	10.811	11.642
5	Dairy Nutrition With Tyari	185	7.027	7.568	7.568
6	Poultry Feeding With Tyari	209	6.220	6.699	6.699
7	Control Of External Parasites With Dominex.	49	6.122	6.122	6.122
8	Poultry Health Management. Practices	446	5.830	8.072	9.193
9	Deworming And Record Keeping In Animal Health	423	5.674	10.875	13.948
10	Poultry Management. Practices	149	5.369	5.369	5.369
11	Why You Should Vaccinate Your Animals	425	5.176	11.059	13.412
12	Diseases In Dairy Farming	739	4.601	5.142	11.096
13	Dairy Health Management	805	4.348	4.845	10.311
14	Poultry Health Management	772	4.275	4.922	10.492

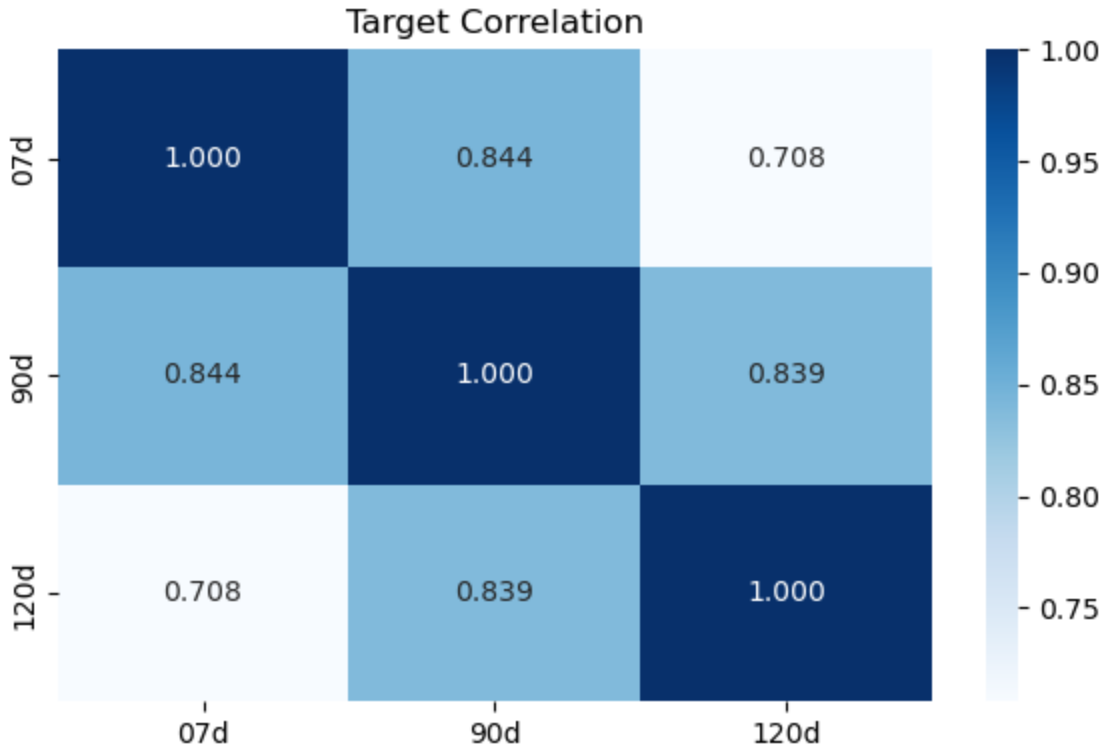


In [129…

```
# ── 3.8 Target Correlation ──────────────────────────────────
# Are the 3 targets correlated? If so, cascade targets can improve predictions.
print('=== TARGET CORRELATION MATRIX ===')
corr = train[TARGETS].corr().round(3)
display(corr)

# Visualise
fig, ax = plt.subplots(figsize=(6, 4))
sns.heatmap(corr, annot=True, fmt='.3f', cmap='Blues', ax=ax,
            xticklabels=['07d','90d','120d'],
            yticklabels=['07d','90d','120d'])
ax.set_title('Target Correlation')
plt.tight_layout()
plt.show()

# Key overlap stats
adopted_07  = train['adopted_within_07_days'] == 1
adopted_90  = train['adopted_within_90_days'] == 1
adopted_120 = train['adopted_within_120_days'] == 1

print('\n=== TARGET OVERLAP ===')
print(f'Adopted at 7d  → also at 90d  : {train.loc[adopted_07,  "adopted_within_90_
print(f'Adopted at 7d  → also at 120d : {train.loc[adopted_07,  "adopted_within_120
print(f'Adopted at 90d → also at 120d : {train.loc[adopted_90,  "adopted_within_120
print(f'Adopted at 120d but NOT at 7d : {train.loc[adopted_120 & ~adopted_07, "ID"]
print('\n→ High overlap = cascade targets is a strong improvement to implement')
```

```
=== TARGET CORRELATION MATRIX ===
```

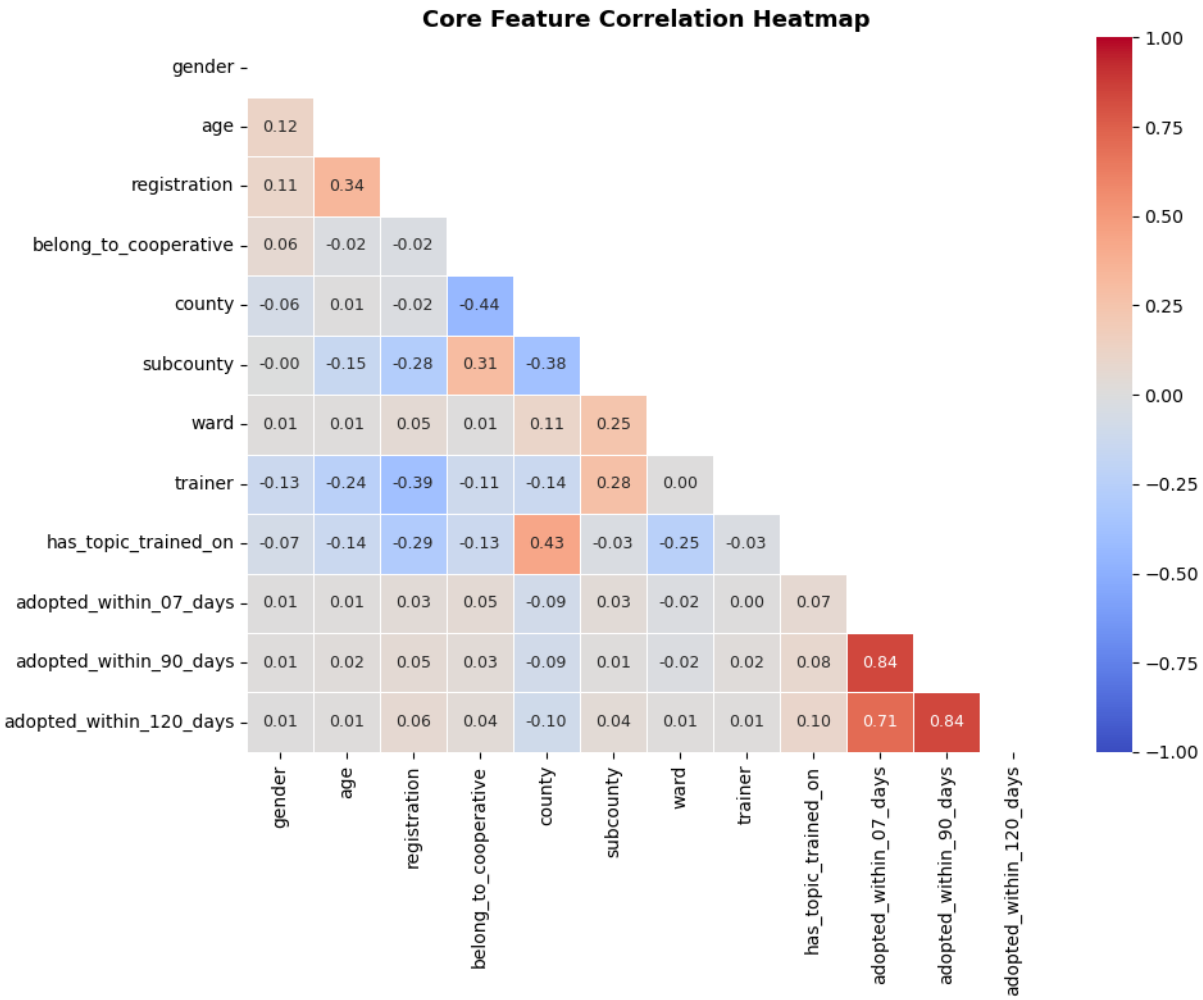
	adopted_within_07_days	adopted_within_90_days	adopted_within_12
adopted_within_07_days	1.000	0.844	
adopted_within_90_days	0.844	1.000	
adopted_within_120_days	0.708	0.839	




```
=== TARGET OVERLAP ===
Adopted at 7d  → also at 90d  : 100.0%
Adopted at 7d  → also at 120d : 100.0%
Adopted at 90d → also at 120d : 100.0%
Adopted at 120d but NOT at 7d : 149 farmers

→ High overlap = cascade targets is a strong improvement to implement
```

In [135…

```
# ── Focused correlation: core features vs targets only ──────
core_cols = ['gender', 'age', 'registration', 'belong_to_cooperative',
             'county', 'subcounty', 'ward', 'trainer',
             'has_topic_trained_on',
             'adopted_within_07_days', 'adopted_within_90_days', 'adopted_within_12

train_corr = train[[c for c in core_cols if c in train.columns]].copy()

le = LabelEncoder()
for col in train_corr.select_dtypes(include='object').columns:
    train_corr[col] = le.fit_transform(train_corr[col].astype(str))

plt.figure(figsize=(10, 8))
corr_matrix = train_corr.corr()
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f',
            cmap='coolwarm', center=0, vmin=-1, vmax=1,
            linewidths=0.5, annot_kws={'size': 9})
plt.title('Core Feature Correlation Heatmap', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



Feature-to-feature correlations (the redundancy check):

registration and **age (0.340):** moderately correlated. Younger farmers tend to register via USSD, older ones manually. Not redundant, just related.

registration and **trainer (-0.387):** notable. Certain trainers tend to attract USSD-registered farmers. Not redundant but worth knowing.

registration and **subcounty (-0.275):** similar pattern, certain subcounties have more USSD farmers.

belong_to_cooperative and **county (-0.442):** the strongest feature-to-feature correlation. Cooperative membership varies significantly by county. This makes geographic sense as some counties have stronger cooperative cultures.

county and **has_topic_trained_on (0.430):** strong. Certain counties have farmers who are more likely to have prior topic training. This is expected since county and trainer are nearly the same thing, and certain trainers are more thorough.

subcounty and **belong_to_cooperative (0.308):** moderate, same geographic cooperative pattern.

In [149…

```
# ── 4.2 Topic Features ──────────────────────────────────────
# parse_topics() is defined in the imports cell — no need to redefine here
from collections import Counter

def engineer_topic_features(df, top_topics, topic_col='topics_list'):
```

```
    """Extract numeric + binary topic features."""
    topics_parsed = df[topic_col].apply(parse_topics)
    df = df.copy()

    # Count/diversity features
    df['topic_total_count']     = topics_parsed.apply(len)
    df['topic_unique_count']    = topics_parsed.apply(lambda x: len(set(x)))
    df['topic_repeat_count']    = df['topic_total_count'] - df['topic_unique_count'
    df['topic_sessions']        = df[topic_col].apply(
        lambda x: len(ast.literal_eval(str(x))) if pd.notna(x) else 0
    )
    df['topic_diversity_ratio'] = df['topic_unique_count'] / (df['topic_total_count

    # Binary flag for each top topic
    for topic in top_topics:
        col_name = 'topic_' + topic.lower().replace(' ','_').replace('(','').replac
        df[col_name] = topics_parsed.apply(lambda x: int(topic in x))

    return df


# Fit top topics on train only — never let test influence which topics are selected
train_topics_parsed = train['topics_list'].apply(parse_topics)
all_topics          = [t for topics in train_topics_parsed for t in topics]
top_topics          = [t for t, _ in Counter(all_topics).most_common(25)]
print(f'Top 25 topics selected for binary features:')
for t, c in Counter(all_topics).most_common(25):
    print(f'  {c:5d}x  {t}')

train = engineer_topic_features(train, top_topics)
test  = engineer_topic_features(test,  top_topics)
print(f'\nTrain shape after topic features: {train.shape}')
```

```
Top 25 topics selected for binary features:
   1657x  The Benefits Of Ndume App
   1294x  Herd Health Management.
   1247x  Biosecurity In Poultry Farming
   1246x  Feeding A Lactating Cow
   1193x  Calf Feeding
   1153x  Poultry Management Practises.
   1091x  Poultry Health Management.
    966x  Dairy Feed Management
    934x  Poultry Diseases And Biosecurity
    930x  Record Keeping In Poultry
    894x  Dairy Health Management
    855x  Milking Hygiene
    855x  Poultry Health Management
    854x  Poultry Management Practices
    850x  How To Succeed In Breeding
    823x  Diseases In Dairy Farming
    823x  Poultry Management Practises..
    821x  Poultry Housing
    806x  Dairy Housing
    788x  Record Keeping In Dairy
    788x  Aflatoxin In Dairy Farming
    785x  Personal Protective Equipment (Ppe)
    736x  Antimicrobial Resistance
    692x  Poultry Health Management..
    632x  Infertility In Dairy Cows

Train shape after topic features: (13536, 45)
```

In [151…

```python
# ── 4.3 Prior Rates from Prior.csv ──────────────────────────
# Compute historical adoption rates per group from Prior.
# Used for: trainer, group_name, county, subcounty, ward

def compute_prior_rates(prior_df, group_col, targets):
    """Compute mean adoption rate and count per group from Prior."""
    agg   = {t: ['mean', 'count'] for t in targets if t in prior_df.columns}
    rates = prior_df.groupby(group_col).agg(agg)
    rates.columns = [f'prior_{group_col}_{t}_{stat}'
                     for t, stat in rates.columns]
    return rates.reset_index()


prior_rate_dfs = {}
for col in ['trainer', 'group_name', 'county', 'subcounty', 'ward']:
    if col in prior.columns:
        rates = compute_prior_rates(prior, col, TARGETS)
        prior_rate_dfs[col] = rates
        print(f'Prior rates for {col}: {rates.shape[0]} groups, {rates.shape[1]-1}

for col, rates_df in prior_rate_dfs.items():
    train = train.merge(rates_df, on=col, how='left')
    test  = test.merge(rates_df,  on=col, how='left')

print(f'\nTrain shape after prior features: {train.shape}')
print(f'Test  shape after prior features: {test.shape}')
```

```
Prior rates for trainer: 9 groups, 6 features
Prior rates for group_name: 840 groups, 6 features
Prior rates for county: 9 groups, 6 features
Prior rates for subcounty: 28 groups, 6 features
Prior rates for ward: 72 groups, 6 features

Train shape after prior features: (13536, 75)
Test  shape after prior features: (5621, 72)
```

In [153…

```python
# ── 4.4 Date Features ──────────────────────────────────────
def engineer_date_features(df, date_col='training_day'):
    df = df.copy()
    df[date_col]        = pd.to_datetime(df[date_col], errors='coerce')
    df['day_of_week']   = df[date_col].dt.dayofweek   # 0=Monday
    df['month']         = df[date_col].dt.month
    df['quarter']       = df[date_col].dt.quarter
    df['is_weekend']    = (df[date_col].dt.dayofweek >= 5).astype(int)
    df['day_of_year']   = df[date_col].dt.dayofyear
    return df

train = engineer_date_features(train)
test  = engineer_date_features(test)
print('Date features added.')
```

```
Date features added.
```

Preprocessing

In [168…

```python
# ── 5.1 Drop identifiers, separate targets ─────────────────
DROP_COLS = ['farmer_name', 'training_day', 'topics_list']

train_ids = train[ID_COL].copy()
test_ids  = test[ID_COL].copy()
y_dict    = {t: train[t].values for t in Targets}

drop_train = DROP_COLS + [ID_COL] + TARGETS
drop_test  = DROP_COLS + [ID_COL]

X_train_df = train.drop(columns=[c for c in drop_train if c in train.columns])
X_test_df  = test.drop(columns=[c for c in drop_test  if c in test.columns])
X_test_df  = X_test_df.reindex(columns=X_train_df.columns)

print(f'Features before encoding: {X_train_df.shape[1]}')
print(f'Remaining categoricals: {X_train_df.select_dtypes(include="object").columns
```

```
Features before encoding: 73
Remaining categoricals: ['gender', 'registration', 'age', 'group_name', 'county', 's
ubcounty', 'ward', 'trainer']
```

In [170…

```python
# ── 5.2 Target encode all high-cardinality columns ──────────
# Replaces each category value with its mean adoption rate.
# Applied to: trainer, group_name, county, subcounty, ward
# Fitted on full train here; will be recomputed per fold inside CV to avoid leakage

TARGET_ENCODE_COLS = ['trainer', 'group_name', 'county', 'subcounty', 'ward']
global_mean = {t: y_dict[t].mean() for t in TARGETS}
```

```
te_maps = {}  # stores encoding maps for use in CV folds

for col in TARGET_ENCODE_COLS:
    if col not in X_train_df.columns:
        continue
    te_maps[col] = {}
    for target in TARGETS:
        temp     = pd.DataFrame({'col': X_train_df[col], 'target': y_dict[target]})
        rate_map = temp.groupby('col')['target'].mean().to_dict()
        te_maps[col][target] = rate_map
        col_name = f'te_{col}_{target}'
        X_train_df[col_name] = X_train_df[col].map(rate_map).fillna(global_mean[tar
        X_test_df[col_name]  = X_test_df[col].map(rate_map).fillna(global_mean[targ
    # Drop original column — replaced by 3 target-encoded versions
    X_train_df = X_train_df.drop(columns=[col])
    X_test_df  = X_test_df.drop(columns=[col])
    print(f'Target encoded: {col} → {[f"te_{col}_{t[-2:]}d" for t in TARGETS]}')

print(f'\nFeatures after target encoding: {X_train_df.shape[1]}')
```

```
Target encoded: trainer → ['te_trainer_ysd', 'te_trainer_ysd', 'te_trainer_ysd']
Target encoded: group_name → ['te_group_name_ysd', 'te_group_name_ysd', 'te_group_na
me_ysd']
Target encoded: county → ['te_county_ysd', 'te_county_ysd', 'te_county_ysd']
Target encoded: subcounty → ['te_subcounty_ysd', 'te_subcounty_ysd', 'te_subcounty_y
sd']
Target encoded: ward → ['te_ward_ysd', 'te_ward_ysd', 'te_ward_ysd']

Features after target encoding: 83
```

In [172…

```
# ── 5.3 Label encode remaining categoricals ──────────────────
# Only low-cardinality columns remain: gender, age, registration
cat_cols = X_train_df.select_dtypes(include=['object','category']).columns.tolist()
print(f'Label encoding: {cat_cols}')

le = LabelEncoder()
for col in cat_cols:
    combined = pd.concat([X_train_df[col], X_test_df[col]]).astype(str).fillna('MIS
    le.fit(combined)
    X_train_df[col] = le.transform(X_train_df[col].astype(str).fillna('MISSING'))
    X_test_df[col]  = le.transform(X_test_df[col].astype(str).fillna('MISSING'))

# Convert booleans
for col in X_train_df.select_dtypes(include='bool').columns:
    X_train_df[col] = X_train_df[col].astype(int)
    X_test_df[col]  = X_test_df[col].astype(int)

# ── 5.4 Impute missing values ────────────────────────────────
# Missing values mainly come from prior rate features where
# a trainer/group in train doesn't appear in prior
imputer = SimpleImputer(strategy='median')
X_train = imputer.fit_transform(X_train_df)
X_test  = imputer.transform(X_test_df)

feature_names = X_train_df.columns.tolist()
print(f'\n✅ Final feature matrix: {X_train.shape}')
```

```
print(f'Features: {feature_names}')
```

```
Label encoding: ['gender', 'registration', 'age']

✅ Final feature matrix: (13536, 83)
Features: ['gender', 'registration', 'age', 'belong_to_cooperative', 'has_topic_trai
ned_on', 'topic_total_count', 'topic_unique_count', 'topic_sessions', 'topic_repeat_
rate', 'topic_the_benefits_of_ndume_app', 'topic_herd_health_management', 'topic_bio
security_in_poultry_farming', 'topic_feeding_a_lactating_cow', 'topic_calf_feeding',
'topic_poultry_management_practises', 'topic_poultry_health_management', 'topic_dair
y_feed_management', 'topic_poultry_diseases_and_biosecurity', 'topic_record_keeping_
in_poultry', 'topic_dairy_health_management', 'topic_milking_hygiene', 'topic_poultr
y_management_practices', 'topic_how_to_succeed_in_breeding', 'topic_diseases_in_dair
y_farming', 'topic_poultry_housing', 'topic_dairy_housing', 'topic_record_keeping_in
_dairy', 'topic_repeat_count', 'topic_diversity_ratio', 'topic_aflatoxin_in_dairy_fa
rming', 'topic_personal_protective_equipment_ppe', 'topic_antimicrobial_resistance',
'topic_infertility_in_dairy_cows', 'prior_trainer_adopted_within_07_days_mean', 'pri
or_trainer_adopted_within_07_days_count', 'prior_trainer_adopted_within_90_days_mean
', 'prior_trainer_adopted_within_90_days_count', 'prior_trainer_adopted_within_120_d
ays_mean', 'prior_trainer_adopted_within_120_days_count', 'prior_group_name_adopted_
within_07_days_mean', 'prior_group_name_adopted_within_07_days_count', 'prior_group_
name_adopted_within_90_days_mean', 'prior_group_name_adopted_within_90_days_count',
'prior_group_name_adopted_within_120_days_mean', 'prior_group_name_adopted_within_12
0_days_count', 'prior_county_adopted_within_07_days_mean', 'prior_county_adopted_wit
hin_07_days_count', 'prior_county_adopted_within_90_days_mean', 'prior_county_adopte
d_within_90_days_count', 'prior_county_adopted_within_120_days_mean', 'prior_county_
adopted_within_120_days_count', 'prior_subcounty_adopted_within_07_days_mean', 'prio
r_subcounty_adopted_within_07_days_count', 'prior_subcounty_adopted_within_90_days_m
ean', 'prior_subcounty_adopted_within_90_days_count', 'prior_subcounty_adopted_withi
n_120_days_mean', 'prior_subcounty_adopted_within_120_days_count', 'prior_ward_adopt
ed_within_07_days_mean', 'prior_ward_adopted_within_07_days_count', 'prior_ward_adop
ted_within_90_days_mean', 'prior_ward_adopted_within_90_days_count', 'prior_ward_ado
pted_within_120_days_mean', 'prior_ward_adopted_within_120_days_count', 'day_of_week
', 'month', 'quarter', 'is_weekend', 'day_of_year', 'te_trainer_adopted_within_07_da
ys', 'te_trainer_adopted_within_90_days', 'te_trainer_adopted_within_120_days', 'te_
group_name_adopted_within_07_days', 'te_group_name_adopted_within_90_days', 'te_grou
p_name_adopted_within_120_days', 'te_county_adopted_within_07_days', 'te_county_adop
ted_within_90_days', 'te_county_adopted_within_120_days', 'te_subcounty_adopted_with
in_07_days', 'te_subcounty_adopted_within_90_days', 'te_subcounty_adopted_within_120
_days', 'te_ward_adopted_within_07_days', 'te_ward_adopted_within_90_days', 'te_ward
_adopted_within_120_days']
```

Model Training

In [181…

```
import lightgbm as lgb

# ── LightGBM parameters ──────────────────────────────────────
LGB_PARAMS = {
    'objective':        'binary',
    'metric':           ['auc', 'binary_logloss'],
    'boosting_type':    'gbdt',
    'n_estimators':     1000,
    'learning_rate':    0.05,
    'max_depth':        6,
    'num_leaves':       31,
```

```
    'min_child_samples': 20,
    'subsample':        0.8,
    'colsample_bytree': 0.8,
    'reg_alpha':        0.1,
    'reg_lambda':       0.1,
    'is_unbalance':     True,   # handles class imbalance natively
    'random_state':     RANDOM_STATE,
    'verbose':          -1,
    'n_jobs':           -1,
}


def train_lgb(X, y, target_name):
    """
    Train LightGBM with 5-fold stratified CV.
    Returns OOF predictions, OOF metrics, and full-data model for test inference.
    """
    print(f'\n{"+"*60}')
    print(f'Target: {target_name}  |  Adoption rate: {y.mean()*100:.2f}%')
    print(f'Positives: {int(y.sum())} / {len(y)}')

    kf        = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDO
    oof_preds = np.zeros(len(y))
    fold_aucs, fold_lls = [], []

    for fold, (tr_idx, val_idx) in enumerate(kf.split(X, y)):
        X_tr, X_val = X[tr_idx], X[val_idx]
        y_tr, y_val = y[tr_idx], y[val_idx]

        model = lgb.LGBMClassifier(**LGB_PARAMS)
        model.fit(
            X_tr, y_tr,
            eval_set=[(X_val, y_val)],
            callbacks=[lgb.early_stopping(50, verbose=False),
                       lgb.log_evaluation(period=-1)]
        )

        preds = model.predict_proba(X_val)[:, 1]
        oof_preds[val_idx] = preds

        auc = roc_auc_score(y_val, preds)
        ll  = log_loss(y_val, preds)
        fold_aucs.append(auc)
        fold_lls.append(ll)
        print(f'  Fold {fold+1}: AUC={auc:.4f}  LogLoss={ll:.4f}  Best iter={model.

    oof_auc = roc_auc_score(y, oof_preds)
    oof_ll  = log_loss(y, oof_preds)
    print(f'  ✅ OOF AUC:     {oof_auc:.4f} ± {np.std(fold_aucs):.4f}')
    print(f'  ✅ OOF LogLoss: {oof_ll:.4f} ± {np.std(fold_lls):.4f}')

    # Retrain on full data using mean best iteration from CV folds
    print('  Retraining on full data...')
    full_params = LGB_PARAMS.copy()
    full_params['n_estimators'] = model.best_iteration_
    full_model = lgb.LGBMClassifier(**full_params)
```

```
    full_model.fit(X, y)

    return oof_preds, oof_auc, oof_ll, full_model


# ── Train with cascading ──────────────────────────────────
# TX_07: no cascade — trained on base features only
# TX_90: add TX_07 OOF as extra feature
# TX_120: add TX_07 + TX_90 OOF as extra features

results  = {}
models   = {}
oof_dict = {}   # stores OOF preds for cascading
pred_dict = {}  # stores test preds for cascading

for i, target in enumerate(TARGETS):
    # Build feature matrix for this target (add cascade features if needed)
    if i == 0:
        X_tr = X_train.copy()
        X_te = X_test.copy()
    elif i == 1:
        # Add TX_07 OOF as feature
        X_tr = np.column_stack([X_train, oof_dict[TARGETS[0]]])
        X_te = np.column_stack([X_test,  pred_dict[TARGETS[0]]])
        print(f'  Added cascade feature: {TARGETS[0]} predictions')
    else:
        # Add TX_07 and TX_90 OOF as features
        X_tr = np.column_stack([X_train, oof_dict[TARGETS[0]], oof_dict[TARGETS[1]]
        X_te = np.column_stack([X_test,  pred_dict[TARGETS[0]], pred_dict[TARGETS[1
        print(f'  Added cascade features: {TARGETS[0]} + {TARGETS[1]} predictions')

    oof, auc, ll, model = train_lgb(X_tr, y_dict[target], target)

    results[target]   = {'oof': oof, 'auc': auc, 'logloss': ll}
    models[target]    = model
    oof_dict[target]  = oof
    pred_dict[target] = model.predict_proba(X_te)[:, 1]
```

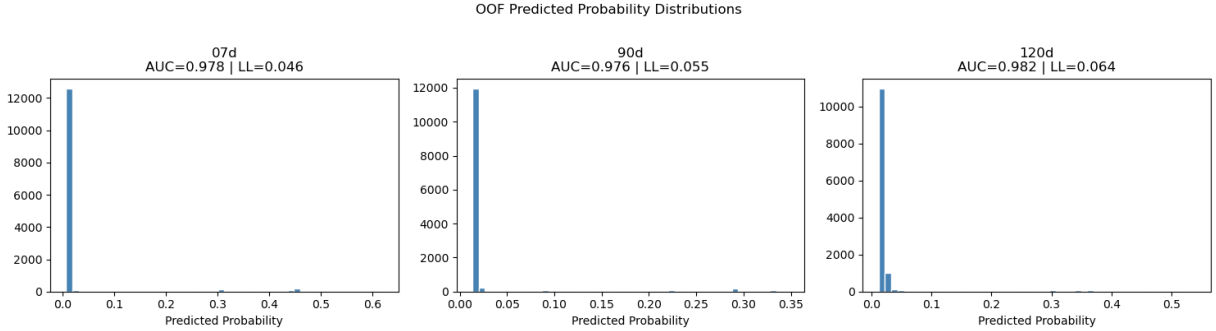
```
============================================================
Target: adopted_within_07_days  |  Adoption rate: 1.13%
Positives: 153 / 13536
  Fold 1: AUC=0.9472  LogLoss=0.0481  Best iter=1
  Fold 2: AUC=0.9634  LogLoss=0.0510  Best iter=1
  Fold 3: AUC=0.9858  LogLoss=0.0439  Best iter=2
  Fold 4: AUC=0.9944  LogLoss=0.0419  Best iter=8
  Fold 5: AUC=0.9797  LogLoss=0.0446  Best iter=1
  ✅ OOF AUC:     0.9783 ± 0.0169
  ✅ OOF LogLoss: 0.0459 ± 0.0032
  Retraining on full data...
  Added cascade feature: adopted_within_07_days predictions

============================================================
Target: adopted_within_90_days  |  Adoption rate: 1.58%
Positives: 214 / 13536
  Fold 1: AUC=0.9855  LogLoss=0.0528  Best iter=3
  Fold 2: AUC=0.9864  LogLoss=0.0527  Best iter=3
  Fold 3: AUC=0.9802  LogLoss=0.0565  Best iter=2
  Fold 4: AUC=0.9421  LogLoss=0.0575  Best iter=2
  Fold 5: AUC=0.9750  LogLoss=0.0572  Best iter=3
  ✅ OOF AUC:     0.9755 ± 0.0164
  ✅ OOF LogLoss: 0.0554 ± 0.0021
  Retraining on full data...
  Added cascade features: adopted_within_07_days + adopted_within_90_days prediction
s

============================================================
Target: adopted_within_120_days  |  Adoption rate: 2.23%
Positives: 302 / 13536
  Fold 1: AUC=0.9880  LogLoss=0.0659  Best iter=6
  Fold 2: AUC=0.9851  LogLoss=0.0655  Best iter=7
  Fold 3: AUC=0.9824  LogLoss=0.0656  Best iter=11
  Fold 4: AUC=0.9880  LogLoss=0.0600  Best iter=12
  Fold 5: AUC=0.9861  LogLoss=0.0627  Best iter=7
  ✅ OOF AUC:     0.9824 ± 0.0021
  ✅ OOF LogLoss: 0.0640 ± 0.0023
  Retraining on full data...
```

In [185…

```
print('\n' + '='*55)
print('📊 CROSS-VALIDATION RESULTS')
print('='*55)
print(f'{"Target":<30} {"AUC":>8} {"LogLoss":>10}')
print('-'*55)
for t in TARGETS:
    print(f'{t:<30} {results[t]["auc"]:>8.4f} {results[t]["logloss"]:>10.4f}')
print('='*55)

# OOF prediction distributions
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for i, t in enumerate(TARGETS):
    axes[i].hist(results[t]['oof'], bins=50, color='steelblue', edgecolor='white')
    axes[i].set_title(f'{t.split("_")[2]}d\nAUC={results[t]["auc"]:.3f} | LL={resul
    axes[i].set_xlabel('Predicted Probability')
plt.suptitle('OOF Predicted Probability Distributions', y=1.02)
plt.tight_layout()
```

```
plt.show()
```

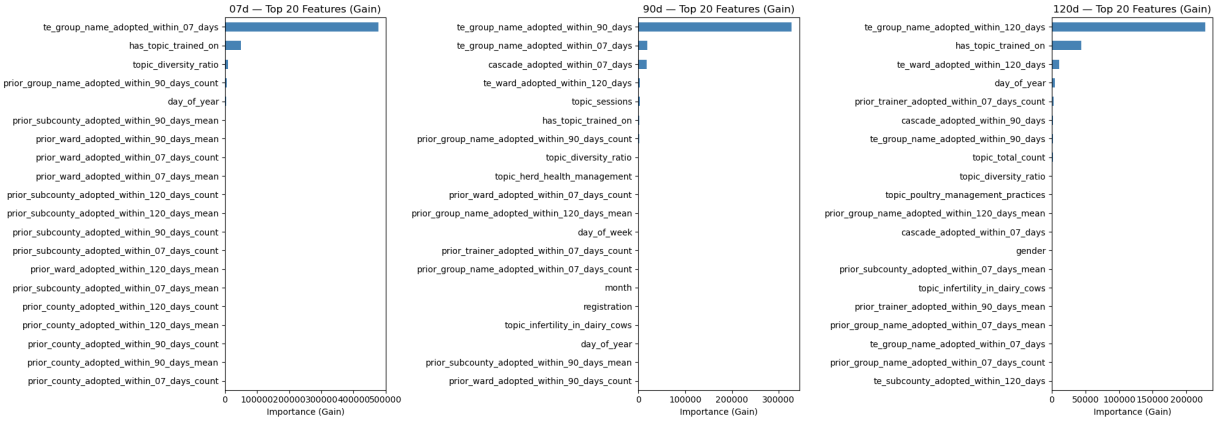
```
============================================================
📊 CROSS-VALIDATION RESULTS
============================================================
Target                                 AUC     LogLoss
------------------------------------------------------------
adopted_within_07_days              0.9783      0.0459
adopted_within_90_days              0.9755      0.0554
adopted_within_120_days             0.9824      0.0640
============================================================
```



Feature Importance

In [190…

```
# LightGBM feature importance
n_features_model = models[TARGETS[0]].n_features_in_
feature_names_model = feature_names[:n_features_model]

fig, axes = plt.subplots(1, 3, figsize=(20, 7))
for i, target in enumerate(TARGETS):
    model = models[target]

    # Build feature names including cascade columns added for this target
    if i == 0:
        names = feature_names.copy()
    elif i == 1:
        names = feature_names + [f'cascade_{TARGETS[0]}']
    else:
        names = feature_names + [f'cascade_{TARGETS[0]}', f'cascade_{TARGETS[1]}']

    imp = pd.Series(
        model.booster_.feature_importance(importance_type='gain'),
        index=names
    )
    imp = imp.sort_values(ascending=False).head(20)
    imp.plot(kind='barh', ax=axes[i], color='steelblue')
    axes[i].invert_yaxis()
    axes[i].set_title(f'{target.split("_")[2]}d — Top 20 Features (Gain)')
    axes[i].set_xlabel('Importance (Gain)')

plt.tight_layout()
plt.show()
```



In [192…

```
# Test predictions are already computed during training (stored in pred_dict)
# Just rename for clarity
test_preds = pred_dict

print('Test prediction summary:')
for target in TARGETS:
    p = test_preds[target]
    print(f'  {target}: mean={p.mean():.4f}  std={p.std():.4f}  min={p.min():.4f}
```

```
Test prediction summary:
  adopted_within_07_days: mean=0.1857  std=0.2139  min=0.0108  max=0.4554
  adopted_within_90_days: mean=0.0343  std=0.0655  min=0.0136  max=0.3164
  adopted_within_120_days: mean=0.0922  std=0.1058  min=0.0157  max=0.4033
```

In [206…

```
# Build submission — using exact column names from sample_sub
col_map = {
    'adopted_within_07_days':  {'auc': 'Target_07_AUC',  'll': 'Target_07_LogLoss'}
    'adopted_within_90_days':  {'auc': 'Target_90_AUC',  'll': 'Target_90_LogLoss'}
    'adopted_within_120_days': {'auc': 'Target_120_AUC', 'll': 'Target_120_LogLoss'
}

submission = pd.DataFrame()
submission['ID'] = test_ids.values

for target, cols in col_map.items():
    probs = np.clip(test_preds[target], 0.001, 0.999)
    submission[cols['auc']] = probs
    submission[cols['ll']]  = probs

# Reorder to match sample_sub exactly
submission = submission[sample.columns]

print(f'Submission shape  : {submission.shape}')
print(f'Columns match     : {list(submission.columns) == list(sample.columns)}')
print(f'Any nulls         : {submission.isnull().any().any()}')
display(submission.head(10))

submission.to_csv('submission.csv', index=False)
print('\n✅ submission.csv saved!')
```

```
Submission shape  : (5621, 7)
Columns match     : True
Any nulls         : False
```

	ID	Target_07_AUC	Target_90_AUC	Target_120_AUC	Target_07_LogLoss	Target_9(
0	ID_LEG1GM	0.455310	0.021146	0.246885	0.455310	
1	ID_1UKOKW	0.010753	0.013609	0.015718	0.010753	
2	ID_U5H2YK	0.010753	0.013609	0.019182	0.010753	
3	ID_55957A	0.010753	0.013609	0.019182	0.010753	
4	ID_N1AC0A	0.010753	0.013609	0.015718	0.010753	
5	ID_4R3SGT	0.010753	0.013609	0.015718	0.010753	
6	ID_NVW38U	0.010753	0.013609	0.017715	0.010753	
7	ID_15LAWX	0.455310	0.021146	0.246885	0.455310	
8	ID_DPNOLZ	0.455310	0.021095	0.246885	0.455310	
9	ID_GXO4MZ	0.010753	0.013609	0.015707	0.010753	

✅ `submission.csv saved!`

In [ ]: